

# Manual de Execução

## Sistema de Controle Preditivo — Digital Twin do Processo Bayer

**Versão:** 1.0 · **Projeto:** projeto\_bayer/ · **Data:** agosto/2026

Este manual descreve, passo a passo, como **instalar**, **configurar** e **executar** o ambiente de supervisão e controle do processo de produção de alumina (Processo Bayer), composto por um **Gêmeo Digital** multicapacidade, um **controlador Fuzzy Adaptativo** e um **agente orquestrado por LangGraph** com intervenção humana obrigatória (HITL).

---

### 1. Visão Geral

O sistema simula a etapa crítica de **controle de nível de decantadores** para evitar transbordamentos, especialmente sob chuva intensa. Ele integra:

- ▢ **Simulação física** dos tanques (série e paralelo) com balanço de volume e química.
- ▢ **Controlador Fuzzy Adaptativo** para abertura modulante das válvulas de drenagem.
- ▢ **Previsão meteorológica** em tempo real (OpenWeather).
- ▢ **Human-in-the-Loop (HITL)** — ações críticas só executam após aprovação do operador.
- ▢ **Persistência** em série temporal (InfluxDB) e **dashboard** interativo (Streamlit).

### Arquitetura

**Fluxo do agente:** coleta → análise → calcular\_controle → [crítico] aguardar\_operador → executar\_controle. Quando o nível (filtrado por EMA) supera o limite crítico sob chuva, o fluxo **interrompe** aguardando aprovação do operador antes de agir fisicamente nas válvulas.

### Visão de sistema (versão implementada)

A descrição completa dos componentes, a comparação com o modelo conceitual e as métricas de validação estão em [docs/SISTEMA\\_IMPLEMENTADO.md](#).

---

### 2. Requisitos

- ▢ **Windows** (o ambiente foi desenvolvido e validado em Windows) — os comandos abaixo são para **PowerShell**.
- ▢ **Python 3.10 ou superior**.
- ▢ **Git** (opcional, para clonar/versionar).
- ▢ Opcionais por funcionalidade:

---

| Funcionalidade | Dependência extra                                |
|----------------|--------------------------------------------------|
| Chuva real     | Conta gratuita no <a href="#">OpenWeatherMap</a> |

| Funcionalidade                 | Dependência extra                                                                         |
|--------------------------------|-------------------------------------------------------------------------------------------|
| Persistência de série temporal | InfluxDB 2.x (local ou via Docker)                                                        |
| Dashboard                      | Pacotes <b>streamlit</b> , <b>plotly</b> , <b>pandas</b> (já em <b>requirements.txt</b> ) |

O **núcleo** (simulador + controladores + agente + testes) roda e é validado **sem** os serviços opcionais — eles degradam com segurança quando indisponíveis.

### 3. Instalação

#### 3.1 Obter o código

```
cd C:\Users\lugos\Documents\Agente_Bayer
# (ou clone do repositório)
git clone https://github.com/LuisGSVasconcelos/bayer-digital-twin.git
cd bayer-digital-twin
```

#### 3.2 Criar o ambiente virtual e instalar dependências

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Se preferir **uv** (mais rápido), o equivalente é:

```
uv venv .venv --seed
uv pip install --python .venv\Scripts\python.exe -r requirements.txt
```

### 4. Configuração (serviços opcionais)

Se quiser chuva real e persistência, crie o arquivo **.env** a partir do modelo:

**Copy-Item .env.example .env**

Preencha com sua chave e token:

```
# OpenWeatherMap
OPENWEATHER_API_KEY=sua_chave_aqui
OPENWEATHER_LAT=-23.5505
OPENWEATHER_LON=-46.6333
OPENWEATHER_UNITS=metric
OPENWEATHER_LANG=pt_br

# InfluxDB
```

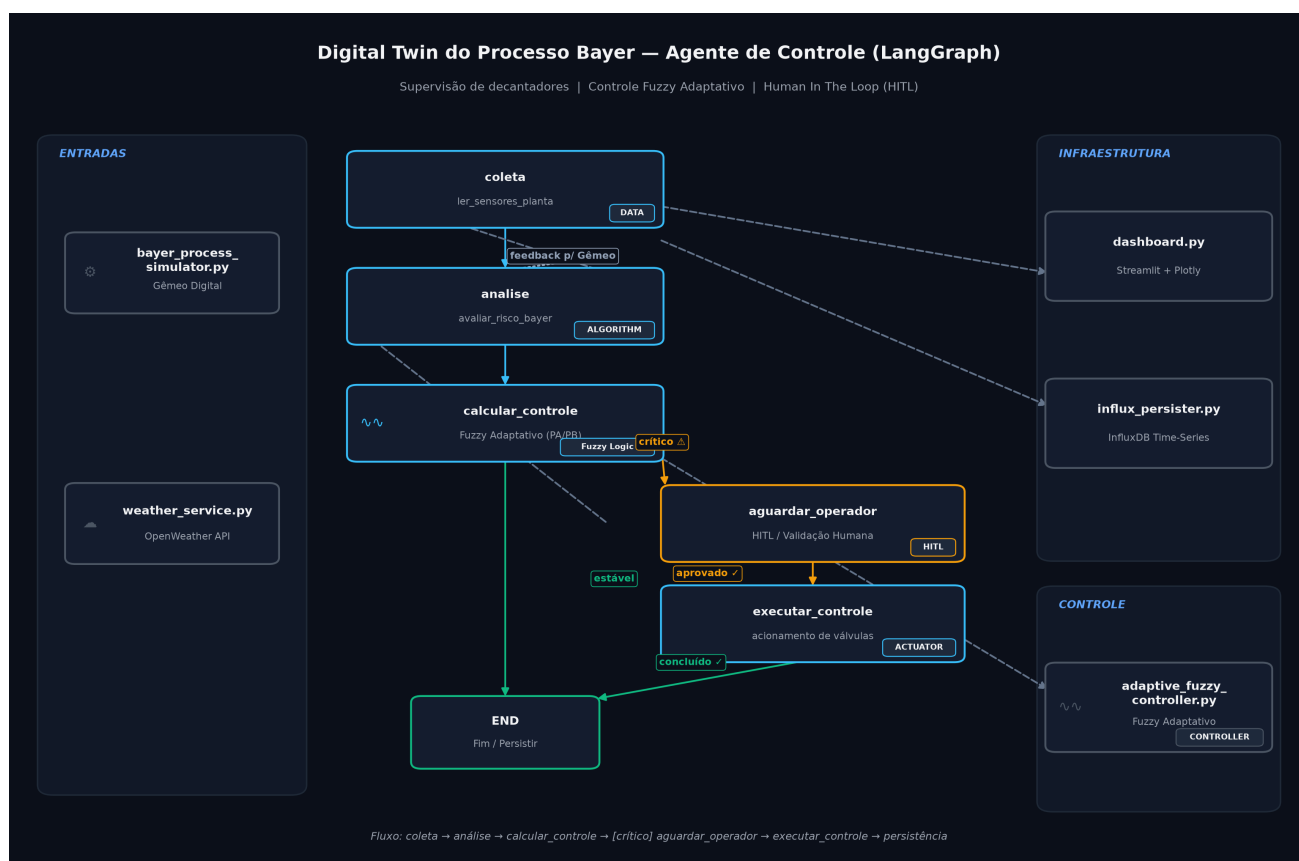


Figure 1: Arquitetura do Digital Twin

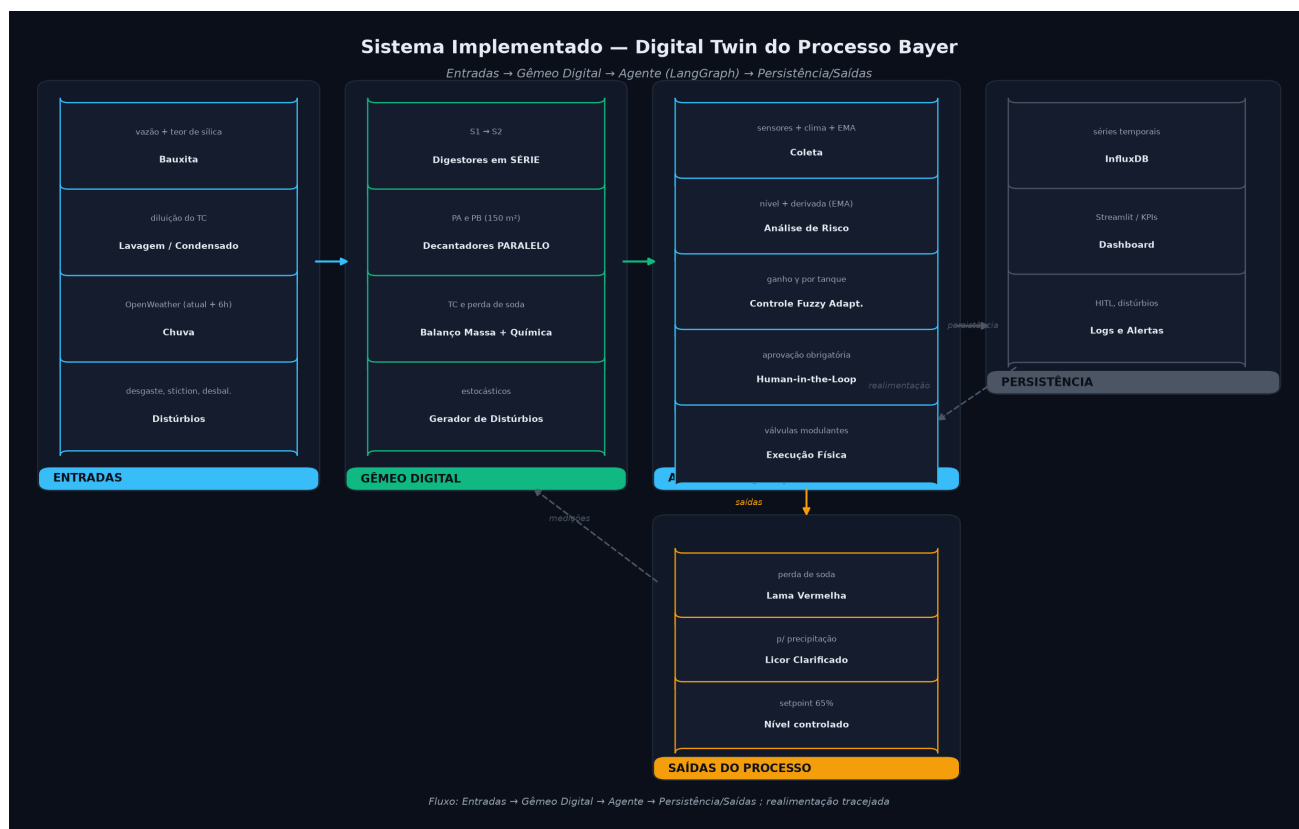


Figure 2: Sistema implementado

```
INFLUXDB_URL=http://localhost:8086
INFLUXDB_TOKEN=seu_token
INFLUXDB_ORG=minha_planta
INFLUXDB_BUCKET=processo_bayer
```

Sem esse arquivo ou sem as credenciais, o sistema roda normalmente usando clima padrão ("sem chuva") e sem persistência — útil para desenvolvimento e testes offline.

---

## 5. Execução

Todas as etapas abaixo assumem o ambiente ativado (item 3.2).

### 5.1 Rodar os testes automatizados

```
pytest -v
```

Esperado: **23 testes aprovados** cobrindo controlador fuzzy, controlador adaptativo, simulador físico e fluxo do agente (HITL), incluindo a correção do domínio do erro.

### 5.2 Benchmark comparativo (Padrão vs Adaptativo)

Cria métrica **reproduzível** (seeds fixas, 10 repetições, média  $\pm$  desvio-padrão):

```
python test_adaptive_controller.py
```

Saída esperada (valores de referência):

```
Fuzzy Padrão:      RMSE 8.14%  $\pm$  0.00
Fuzzy Adaptativo: RMSE 7.81%  $\pm$  0.00
Melhoria: +4.1%
```

### 5.3 Executar o agente em modo texto (terminal)

Executa ciclos de supervisão e, quando detecta risco crítico, **pausa pedindo aprovação**:

```
python langgraph_agent.py
```

Em caso de alerta, o console pergunta **Autorizar abertura emergencial das válvulas? (s/n)**. Digite **s** para liberar a ação ou **n** para abortar.

### 5.4 Dashboard interativo (Streamlit)

```
streamlit run dashboard.py
```

Abra no navegador: **http://localhost:8501**

No painel: - clique **Iniciar** para rodar a simulação e **Parar** para pausar; - no seletor **Modo de simulação**, escolha ► **Contínuo** (roda sozinho com **Iniciar/Parar**) ou ☐ **Manual (ticks)** (está pausado e avança **exatamente 1, 10 ou 30 ciclos** a cada clique) —

ideal para acompanhar o sistema ciclo a ciclo; - o **tempo é fictício e independente do relógio do computador**: cada ciclo de processo conta 1 s a partir de uma época fixa (2024-01-01 00:00), então o eixo do tempo avança de forma reprodutível em ambos os modos (contínuo e manual); - os gráficos mostram **nível/abertura (PA e PB), química e distúrbios** (TC, perda de soda, chuva) e a **vazão de alimentação** (L/s, com a oscilação do distúrbio “Variação de alimentação”); - ajuste o **movimento** (ciclos por atualização) — a página usa **renderização por fragmento**, atualizando só os gráficos/status a cada ~0,4 s sem piscar a página inteira; - no seletor **Cenário de clima**, force chuva fixa (**Forte/Moderada/Nenhuma**), ajuste a **chuva de forma contínua** com o **slider “Manual...”** (0–0,30 mm/s), ou use a **API real**; - no seletor ☐ **Controlador**, escolha a **estratégia de controle**: **PI (drenagem + makeup)** — segura o setpoint de forma bidirecional — ou **Fuzzy Adaptativo** — controlador fuzzy com ganho adaptativo atuando **tanto na drenagem quanto no makeup** (bidirecional, com **faixa morta + limite de slew** para modular sem estalar), para **comparar** o comportamento (no teste de referência com chuva forte: PI mín 64,98 / mws 65,39; fuzzy mín 64,47 / mws 64,88); - quando o alerta de HITL aparecer, o loop **pausa** (não congela) e abre o botão ☐ **Aprovar Ação Emergencial** no painel **HITL**; clique-o para liberar a válvula; - no painel ☐ **Distúrbios**, **ligue/desligue** cada distúrbio (variação de alimentação, desgaste da bomba, atrito da válvula, desbalanceamento PA/PB, diluição de TC, sílica e picos de sensor) — **todos ficam ativos por padrão**, mas você pode desativar os que quiser; - no painel ☐ **Válvula de saída (manual)**, marque **“Atuação manual da válvula”** para **definir a abertura PA/PB manualmente** (sliders 0–100%), sobrepondo o controle PI — útil para testar a resposta do nível a um comando manual (desligue p/ voltar ao automático); - quando uma **ação emergencial** dispara, o loop **pausa** e aparece um **banner + botão “Aprovar” em destaque no corpo** (não é travamento) — aprovar libera a ação; em **modo manual** da válvula, o HITL pendente é **liberado automaticamente** (o operador já está acionando a válvula); - acompanhe KPIs (níveis, TC, perda de soda) e gráficos (PV × SP × MV, química, distúrbios).

**Comportamento do controle:** o agente atua como **regulador PI bidirecional**: quando o nível fica **acima** do setpoint (**65%**) abre a válvula de **drenagem**; quando fica **abaixo**, abre a **água de reposição (makeup)**, que injeta fluxo e repõe o nível. Isso **corrige** o problema do atuador só-drenagem (que não conseguia levantar o nível) — o setpoint é **sustentado mesmo sem chuva**. A modulação é **suave** (bandas proporcionais + integrais por sentido, com anti-windup), sem oscilar 0–100%. O **HITL** gateia apenas a **emergência** (nível **>80%** ou **>70% com tendência de alta + chuva**): nessa condição o loop **pausa** (gráficos/log param — espera correta) e mostra o botão **Aprovar**. Ao aprovar (uma vez), o controle traz o nível de volta ao setpoint e o **segura**.

**Balanço do decantador:** corrigido para que a **chuva e a alimentação afetem o nível** — o licor clarificado sai apenas o licor **separado** (sem a chuva); a chuva **acumula** no decantador e eleva o nível. O controle então **abre a válvula proporcionalmente à chuva** para manter os 65%. O balanço também **conserva a caustica** (a soda que sai no licor clarificado é abatida da massa), de modo que o **TC estabiliza** perto do valor de entrada, com uma perda química de sílica **realista** (pequena).

**Nota (makeup × nível):** o controle é **bidirecional** — acima do setpoint usa a drenagem, abaixo usa a **água de reposição (makeup)**, que injeta volume e **sustenta o nível mesmo sem chuva**. Por isso o setpoint é mantido em qualquer condição de chuva (Forte, Moderada, Nenhuma ou Manual). O makeup

aparece no gráfico de controle como linha verde (MV).

**Cenário de demonstração:** o dashboard abre com os decantadores **acima do limiar ( $\approx 80,5\%$ )** — o HITL dispara já no primeiro ciclo (botão Aprovar disponível). Ao aprovar, a ação é **travada**: a válvula continua drenando **sem re-pedir aprovação a cada ciclo** até o nível sair do estado crítico, chegando ao setpoint ( $\sim 65\%$ ) e se mantendo lá (a válvula mantém uma abertura para compensar a chuva). O demo ativa apenas os **distúrbios de química** (sílica e diluição de TC), de modo que a **perda de soda e o TC variam** com a alimentação constante — que, por sua vez, mantém o **nível estável no setpoint**. O conjunto completo de distúrbios fica ativo no benchmark (`test_adaptive_controller.py`) e no simulador. A simulação roda leve ( $\sim 60$  ms/tick) e os **spikes de sensor são desabilitados** no demo.

---

## 6. Estrutura de Arquivos

```
projeto_bayer/
├── bayer_process_simulator.py # Gêmeo Digital (tanques, decantadores, distúrbios)
├── fuzzy_controller.py        # Controlador Fuzzy base (Mamdani + centroide)
├── adaptive_fuzzy_controller.py # Fuzzy com ganho adaptativo (gradiente + momentum)
├── weather_service.py         # Integração OpenWeather (opcional)
├── langgraph_agent.py         # Agente LangGraph + HITL
├── influx_persister.py        # Persistência InfluxDB (opcional)
├── dashboard.py               # Dashboard Streamlit + Plotly
├── test_adaptive_controller.py # Benchmark reproduzível
├── tests/                     # Suíte pytest (23 testes)
├── docs/
│   └── SISTEMA_IMPLEMENTADO.md # Visão completa do sistema (componentes, métricas)
├── scripts/
│   ├── diagrama_arquitetura.py # Gera o diagrama do fluxo (PNG dark-mode)
│   └── diagrama_sistema_implementado.py # Gera o diagrama em subgrafos (PNG dark-
mode)
├── assets/
│   ├── arquitetura_processo_bayer.png
│   └── arquitetura_sistema_implementado.png
├── requirements.txt
├── .env.example               # Modelo de configuração de serviços
└── MANUAL_EXECUCAO.md        # Este documento
```

---

## 7. Solução de Problemas (Troubleshooting)

| Sintoma                      | Provável causa                          | Solução                                                            |
|------------------------------|-----------------------------------------|--------------------------------------------------------------------|
| <b>pytest</b> não encontrado | Ambiente não ativado /<br>deps ausentes | Ative o venv e rode <b>pip<br/>install -r<br/>requirements.txt</b> |

| Sintoma                                   | Provável causa                                    | Solução                                                                                   |
|-------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------|
| Erro ao importar <b>influxdb_client</b>   | Pacote não instalado                              | <b>pip install influxdb-client</b><br>(opcional; sem ele a persistência vira no-op)       |
| Clima sempre "sem chuva" / "indisponível" | Sem chave OpenWeather ou sem <b>.env</b>          | Preencha <b>OPENWEATHER_API_KEY</b> no <b>.env</b>                                        |
| Dashboard não inicia                      | <b>streamlit/plotly</b> ausentes, ou porta em uso | <b>pip install streamlit plotly pandas</b> ; troque a porta com <b>--server.port 8502</b> |
| Branch <b>master</b> vs <b>main</b>       | Inicialização Git                                 | Envie com <b>git push -u origin main</b>                                                  |
| Autenticação GitHub pedindo senha         | Token ausente                                     | Usar Windows Credential Manager ou token classic com escopo <b>repo</b>                   |

## Regenerar o diagrama de arquitetura

```
python scripts\diagrama_arquitetura.py
```

## 8. Referência Rápida de Comandos

```
# instalação
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# configuração (opcional)
Copy-Item .env.example .env

# validação
pytest -v
python test_adaptive_controller.py

# aplicação
streamlit run dashboard.py           # dashboard web
python langgraph_agent.py           # agente em modo texto (HITL interativo)
```

Documento gerado como artefato do projeto **projeto\_bayer** — Processo Bayer (alumina).